

Cyclic Dependency in Spring Boot

What is Cyclic Dependency?

A cyclic (circular) dependency happens when two Spring beans depend on each other for creation.

Bean A → needs Bean B

Bean B → needs Bean A

Spring cannot decide which one to create first.

This is not a runtime problem — it is a **startup-time failure**.

How It Happens Internally

Spring creates beans like this:

1. Create Bean A
2. Inject dependencies into Bean A
3. If Bean A needs Bean B → create Bean B
4. If Bean B needs Bean A → create Bean A again
5. Spring detects a loop → throws exception

This is why circular dependencies crash applications **before the server starts**.

Real Code Example (Problem)

StudentController

```
@RestController
public class StudentController {

    @Autowired
    private Student student;

    public void notifyStudent() {
        System.out.println("Notifying student...");
    }
}
```

Student

```
@Component
public class Student {

    @Autowired
    private StudentController studentController;

    public void update() {
        studentController.notifyStudent();
    }
}
```

What Spring sees

```
StudentController → Student
Student → StudentController
```

This forms a cycle.

Spring throws:

BeanCurrentlyInCreationException

```
BeanCurrentlyInCreationException
Requested bean is currently in creation
```

Fix 1 — @Lazy Injection

```
@Autowired
@Lazy
private StudentController studentController;
```

What really happens here?

Spring does NOT inject the real StudentController.

It injects a **proxy object** instead.

That proxy:

- Looks like StudentController
- Does nothing until a method is actually called

So Spring can:

1. Create Student
2. Inject proxy
3. Create StudentController
4. Later replace proxy with real object

This breaks the creation loop.

⚠ But the design is still wrong — only delayed.

Fix 2 — Setter Injection

```
@Component
public class Student {

    private StudentController controller;

    @Autowired
    public void setController(StudentController controller) {
        this.controller = controller;
    }
}
```

Why this works

Spring can:

1. Create both objects first (without wiring)
2. Then inject dependencies afterward

This avoids circular construction.

Better than @Lazy, but still not ideal.

Fix 3 — allow-circular-references (Worst)

Add the below line in Application Properties file

```
spring.main.allow-circular-references=true
```

Spring will force the app to start —
but the design problem still exists.

Never use this in real systems.

Fix 4 — Correct Design (Best Practice)

Move shared logic into a **Service**.

StudentService

```
@Service
public class StudentService {
    public void notifyStudent() {
        System.out.println("Notifying student");
    }
}
```

StudentController

```
@Autowired
private StudentService service;
```

Student

```
@Autowired
private StudentService service;
```

Now:

1. Controller → Service
2. Student → Service
3. No circular reference

This is how real enterprise applications are built.

Interview Gold Line

“Circular dependency happens when two beans depend on each other during creation. It should be solved using better design, not configuration tricks.”